

Sapientia Erdélyi Magyar Tudományegyetem
Marosvásárhelyi Kar
Számítástechnika szak

Bus4U - Szerver oldal

DIPLOMADOLGOZAT

Témavezető:
Dr. Szántó Zoltán

Hallgató:
Zsigmond Attila

2024

Universitatea „Sapientia” din Cluj-Napoca
Facultatea de Științe Tehnice și Umaniste din Târgu-Mureș
Specializarea Calculatoare

Bus4U - Partea de Server

PROIECT DE DIPLOMĂ

Coordonator științific:
Dr. Szántó Zoltán

Student:
Zsigmond Attila

2024

Kivonat

Scopul aplicației web este de a face sistemele moderne de transport public mai prietenoase cu utilizatorul și mai eficiente, cu un accent deosebit pe planificarea călătoriilor, achiziționarea de bilete și urmărirea autobuzelor. Aplicația permite utilizatorilor să își planifice călătoriile în avans în cadrul rețelei de transport public dintr-un oraș sau o regiune specifică. Utilizatorii își pot achiziționa biletele pentru o rută selectată și pot vizualiza orele de plecare și sosire ale curselor, defalcat pe orașe și stații, asigurând astfel flexibilitate și informații de călătorie precise.

Aplicația oferă, de asemenea, o hartă interactivă care permite urmărirea în timp real a pozițiilor curente ale autobuzelor. Această funcție este deosebit de utilă atunci când apar schimbări neprevăzute în program, deoarece utilizatorii pot fi informați la timp despre întârzieri sau modificări ale traseelor. Pe lângă urmărirea în timp real, sistemul oferă locațiile geografice precise ale stațiilor de autobuz individuale, pe care utilizatorii le pot filtra în funcție de orașe sau rute, facilitând astfel planificarea la care stație să ajungă.

O aplicație separată care rulează pe terminalele POS aflate pe autobuze se ocupă de validarea biletelor și de partajarea continuă a pozițiilor autobuzelor cu serverul central. Această funcție asigură că toți participanții la sistem au informații actualizate, permițând utilizatorilor să rămână informați despre locațiile în timp real ale autobuzelor.

O altă componentă importantă a aplicației este interfața de administrare utilizată de companiile de autobuze. Această interfață permite gestionarea programelor, adăugarea de noi rute și stații și actualizarea datelor existente. În plus, informațiile despre angajați, inclusiv detaliile șoferilor și ale altui personal, precum și datele tehnice și de operare ale autobuzelor, pot fi gestionate aici. Acest sistem de administrare asigură funcționarea eficientă și întreținerea programelor, permițând companiilor de autobuze să își actualizeze cu ușurință serviciile și să asigure acuratețea.

Tema centrală a lucrării de diplomă este descrierea detaliată a componentelor serverului aplicației web, care asigură gestionarea fluidă a datelor utilizatorilor, a tranzacțiilor de achiziție a biletelor, a informațiilor de program și a urmăririi autobuzelor în timp real.

Cuvinte cheie: transport public, urmărire în timp real, aplicație web, Rails API

Kivonat

The purpose of the web application is to make modern public transportation systems more user-friendly and efficient, with a particular focus on trip planning, ticket purchasing, and bus tracking. The application allows users to plan their trips in advance within a specific city or region's public transportation network. Users can purchase tickets for a selected route and view the departure and arrival times of the services, broken down by cities and stops, thus ensuring flexibility and accurate travel information.

The application also offers an interactive map that allows real-time tracking of the current positions of buses. This feature is especially useful when unexpected changes occur in the schedule, as users can be timely informed about delays or route changes. In addition to real-time tracking, the system provides the precise geographical locations of individual bus stops, which users can filter based on cities or routes, making it easier to plan which stop to arrive at.

A separate application running on the POS terminals located on the buses ensures ticket validation and the continuous sharing of the buses' positions with the central server. This feature ensures that all participants in the system have up-to-date information, allowing users to stay informed about the real-time locations of the buses.

Another important component of the application is the administration interface used by the bus companies. This interface allows for the management of schedules, the addition of new routes and stops, and the updating of existing data. Additionally, employee information, including details of drivers and other staff, as well as technical and operational data of the buses, can be managed here. This administration system ensures efficient operations and schedule maintenance, enabling bus companies to easily update their services and ensure accuracy.

The central theme of the thesis is the detailed description of the server-side components of the web application, which ensure the seamless management of user data, ticket purchase transactions, schedule information, and real-time bus tracking.

Keywords: public transportation, real-time tracking, web application, Rails API

Kivonat

A webalkalmazás célja a modern tömegközlekedési rendszerek felhasználóbaráttá és hatékonyabbá tétele, különös tekintettel az utazás megtervezésére, jegyvásárlásra, valamint az autóbuszok követésére. Az alkalmazás lehetővé teszi a felhasználók számára, hogy előre megtervezhessék utazásaikat egy adott város vagy régió tömegközlekedési hálózatán belül. A felhasználók megvásárolhatják jegyeiket egy kiválasztott járatra, valamint megtekinthetik a járatok indulási és érkezési időpontjait, városokra és megállókra bontva, biztosítva ezzel a rugalmasságot és a pontos utazási információkat.

Az alkalmazás egy interaktív térképet is kínál, amelyen valós időben nyomon követhetők az autóbuszok jelenlegi pozíciói. Ez a funkció különösen hasznos, amikor a menetrendben váratlan változások történnek, mivel a felhasználók időben értesülhetnek a késésekről vagy útvonal-módosításokról. A valós idejű követés mellett a rendszer biztosítja az egyes buszmegállók pontos földrajzi helyzetét, amelyeket a felhasználók városok vagy útvonalak alapján szűrhetnek, így könnyebben tervezhetik meg, melyik megállóhoz érkezenek.

A buszokon található POS terminálokon futó különálló alkalmazás gondoskodik a jegyérvényesítésről és az autóbuszok pozíciójának folyamatos megosztásáról a központi szerverrel. Ez a funkció biztosítja, hogy a rendszer minden résztvevője naprakész információval rendelkezzen, így a felhasználók folyamatosan tájékozódhatnak az autóbuszok valós helyzetéről.

Az alkalmazás másik fontos része az adminisztrációs felület, amelyet a busztársaságok használhatnak. Ezen a felületen lehetőség van a menetrendek kezelésére, új útvonalak és megállók felvitelére, valamint a meglévő adatok frissítésére. Továbbá itt kezelhetők az alkalmazottak információi, beleértve a sofőrök és egyéb személyzet adatait, valamint az autóbuszok műszaki és üzemeltetési adatait. Ez az adminisztrációs rendszer biztosítja a hatékony működést és a menetrendek karbantartását, így a busztársaságok könnyedén frissíthetik szolgáltatásaikat és biztosíthatják a pontosságot.

A dolgozat központi témája a webalkalmazás szerver oldali komponenseinek részletes ismertetése, amelyek biztosítják a felhasználói adatok, a jegyvásárlási tranzakciók, a menetrendi információk, valamint a valós idejű autóbusz-követés zökkenőmentes kezelését.

Kulcsszavak: tömegközlekedés, valós idejű követés, webalkalmazás, Rails API

Tartalomjegyzék

1. Bevezető	5
2. Célkitűzések	6
3. Elméleti megalapozás és bibliográfiai tanulmány	7
3.1. Technológiai háttér	7
3.1.1. Ruby on Rails	7
3.1.2. PostgreSQL	8
3.1.3. Azure	8
3.2. A közlekedési rendszerek és a jövőbeli trendek	9
3.3. Hasonló alkalmazások	9
3.3.1. Felhasználói élmény és innovációk	10
3.3.2. Hatások a közlekedési rendszerre	10
4. Rendszer specifikációi	11
4.1. Felhasználói követelmények	11
4.1.1. Felhasználói weboldal	11
4.1.2. Admin felület	11
4.1.3. Mobil applikáció	12
4.1.4. POS terminál alkalmazás	12
4.2. Rendszerkövetelmények	13
4.2.1. Funkcionális követelmények	13
4.2.2. Nem-funkcionális követelmények	14
5. A rendszer részletes leírása	16
5.1. Az API felépítése	17
5.2. Endpointok részletes leírása	17
5.2.1. GET /get_stations_by_city	17
5.3. Adatkezelés és adatbázis kapcsolat	18
5.3.1. Adatbázis táblák	18
5.4. Projektmenedzsment és verziókövetés	20

5.4.1.	Jira	21
5.4.2.	Git és GitHub	22
6.	Infrastruktúra	24
6.1.	Azure infrastruktúra áttekintése	24
6.2.	Domain konfiguráció	24
6.3.	Adatbázis telepítése és konfigurációja	24
6.4.	Szerver telepítése	25
7.	Az alkalmazás működése	26
7.1.	Felhasználói felület	26
7.1.1.	Főoldal	26
7.1.2.	Jegyek	29
7.1.3.	Menetrend	30
7.1.4.	Megállók és térkép	31
7.2.	Adminisztrációs felület	32
7.2.1.	Kezdőlap	32
7.2.2.	Megállók	32
7.2.3.	Útvonalak	33
7.2.4.	Menetrend	33
7.2.5.	Buszok és alkalmazottak	34
7.3.	POS terminál alkalmazása	35
7.3.1.	GPS oldal	35
7.3.2.	Jegyszkennelő oldal	35
8.	A rendszer tesztelése	37
8.1.	Tesztelési környezet	37
8.2.	Funkcionális tesztelés	37
8.2.1.	Végpontok tesztelése	38
8.2.2.	Tesztelési példa	38
8.2.3.	Tesztelési eredmények	39
8.3.	Teljesítménytesztelés	39
8.3.1.	Teljesítmény-metrikák	39
8.4.	Biztonsági tesztelés	39
8.5.	Összegzés	40
9.	Összefoglalás	41
	Irodalomjegyzék	42
A	Függelék	44

Ábrák jegyzéke

4.1. Use-case diagram a felhasználók és az adminisztrátorok lehetőségeiről . . .	13
5.1. A rendszer architektúra diagramja	17
5.2. Az rendszer adatbázisdiagramja	20
5.3. A Jira kanban board áttekintése	22
5.4. A GitHub repository áttekintése	23
6.1. Telepítési diagram	25
7.1. Aktivitás diagram az utazás tervezésének folyamatáról	27
7.2. Megvásárolt jegy	28
7.3. Szekvencia diagram az útvonaltervezés folyamatáról	28
7.4. Felhasználó megvásárolt jegyei	29
7.5. Szekvencia diagram a jegyvásárlás folyamatáról	30
7.6. Listázott menetrend és az útvonal térképe	31
7.7. Szekvencia diagram a dokumentumok ellenőrzéséről	32
7.8. Megállók létrehozása/törlése	33
7.9. Menetrend létrehozása/törlése	34
7.10. Buszok és alkalmazottak kezelése	34
7.11. Busz kiválasztása és GPS követés	35
7.12. Jegy szkennelése	36

1. fejezet

Bevezető

A mai világunk velejárója, hogy egyre több dolgot digitalizálunk az életünk egyszerűsítésének érdekében. Az autóbuszok használata a közlekedés egyik legkörnyezetkímélőbb formája, de sajnos sok hátránnyal is jár. Ezt otthon is megfigyeltük, de amióta egyetemre járunk, itt Marosvásárhelyen is megtapasztaltuk a tömegközlekedés hiányosságait. Ezek közül a legkiemelkedőbb a kártyás fizetés lehetőségének a hiánya. Újonnan a városi buszokon adódik már erre lehetőség, ám a hosszútávú utazásoknál még mindig csak készpénzzel lehet fizetni.

Jelenleg arra sincs lehetőség, hogy egy utazásra előre jegyet vásároljunk. Egy másik nagy probléma, hogy nem minden megállóban elérhető a buszok közlekedésének menetrendje, illetve ezek online sehol nem tekinthetők meg. Emellett Vásárhelyen a legnagyobb problémának a pontos közlekedés hiánya tekinthető szerintünk. Ha egy adott megállóban ki is van írva, hogy mikor közlekednek a buszok, sok esetben vagy késnek, vagy rosszabb esetben hamarabb elindulnak. Kiaknázva a technológia által nyújtott lehetőségeket, szeretnénk egy olyan alkalmazást létrehozni, amely segíti a tömegközlekedés népszerűsítését, és egyszerűbbé teszi a buszok használatát.

Busztársaságok szempontjából is fontos a digitalizáció. A profitot és a költségeket egyszerűbb és átláthatóbb egy online térben számon tartani, mint papíron. A buszok állapotát, papírjainak lejáratát is hasznos online követni, mivel így elkerülhetők az esetleges büntetések.

Úgy gondoljuk, hogy sok embert tudnánk motiválni a tömegközlekedés használatára, ha a fenti problémákat egy alkalmazással meg tudnánk oldani, javítva a busztársaságok, és a felhasználók élményét is.

Emellett kiemelten fontosnak tartjuk a környezetvédelmet is, hiszen a buszközlekedés támogatása hozzájárul a fenntartható városi közlekedéshez. A digitális megoldások alkalmazásával nemcsak a felhasználói élményt javíthatjuk, hanem hozzájárulhatunk a városok környezetbarát fejlődéséhez is.

2. fejezet

Célkitűzések

A projekt célja, hogy egy modern, felhasználóbarát webalkalmazást fejlesszen ki a tömegközlekedés népszerűsítésére és a buszok használatának egyszerűsítésére. A következő konkrét célkitűzéseket határozzuk meg:

- **Digitális Jegyvásárlás:** Olyan funkció kialakítása, amely lehetővé teszi a felhasználók számára, hogy előre jegyet vásároljanak a kívánt járatra, így csökkentve a készpénzes tranzakciók számát.
- **Valós Idejű Buszkövetés:** Integrálni egy valós idejű buszkövető rendszert, amely lehetővé teszi a felhasználók számára, hogy nyomon követhessék a buszok aktuális helyzetét és várható érkezési idejét.
- **Menetrendek Elérhetősége:** Biztosítani, hogy a buszmenetrendek minden megállóban könnyen elérhetők legyenek, valamint online platformon is megtekinthetők legyenek.
- **Interaktív Térkép:** Fejleszteni egy interaktív térképet, amely lehetővé teszi a felhasználók számára, hogy könnyen navigáljanak a város tömegközlekedési hálózatán, és információkat kapjanak a megállókról és járatokról.
- **Adminisztrációs Felület:** Létrehozni egy adminisztrációs felületet, amelyet a busztársaságok használhatnak a menetrendek, útvonalak és alkalmazottak információinak kezelésére, ezzel növelve a működés hatékonyságát.
- **Felhasználói Élmény Javítása:** A webalkalmazás tervezése során a felhasználói élmény maximalizálása, beleértve a könnyű navigációt és az intuitív felület kialakítását.

Ezek a célkitűzések a webalkalmazás fejlesztésének alapját képezik, és hozzájárulnak a modern tömegközlekedési rendszerek hatékonyabbá tételéhez, valamint a felhasználók utazási élményének javításához.

3. fejezet

Elméleti megalapozás és bibliográfiai tanulmány

3.1. Technológiai háttér

A Bus4U alkalmazás digitális megoldásai mögött modern webtechnológiák állnak, amelyek lehetővé teszik a felhasználói élmény folyamatos fejlesztését és a szolgáltatások hatékony működését. Az alkalmazás backendje Ruby on Rails keretrendszert használ, amely az egyik legnépszerűbb választás webes alkalmazások fejlesztésére.

3.1.1. Ruby on Rails

A *Ruby on Rails* egy népszerű és széles körben használt webes keretrendszer, amely a Ruby programozási nyelvre épül. A Rails két alapvető elv alapján működik: a „*Convention over Configuration*” és a „*Don't Repeat Yourself*”. Ezek az elvek azt eredményezik, hogy a fejlesztőknek kevesebb kóddal kell dolgozniuk, mivel a Rails konvenciói automatikusan meghatározzák a konfigurációkat, így csökkentve a redundáns kódot és növelve a fejlesztési folyamat hatékonyságát [1, 2]. Ez különösen hasznos nagyobb projektek esetén, mivel elősegíti a gyorsabb fejlesztést és az egyszerűbb karbantartást, amely a gyorsan változó üzleti igények mellett nélkülözhetetlen.

A Rails *MVC (Model-View-Controller)* architektúrára épül, amely logikailag elkülöníti az alkalmazás különböző részeit: az adatkezelést (Model), a felhasználói felületet (View), és az üzleti logikát (Controller). Ez a struktúra átláthatóbbá teszi a kódot, mivel a különböző funkciók külön-külön karbantarthatók, anélkül hogy zavarnák egymást, valamint megkönnyíti az új funkciók hozzáadását is. A Rails alkalmazásokhoz számos „*gem*”, vagyis könyvtár érhető el, amelyek előre elkészített megoldásokat kínálnak, így még a komplexebb funkciók integrálása is könnyebb és biztonságosabb lehet [1].

A Rails további előnye, hogy natív módon támogatja *RESTful API*-k létrehozását.

A RESTful megközelítés egyértelmű, szabványosított formát biztosít az API-k számára, amely megkönnyíti az alkalmazások közötti integrációt, például egy mobilalkalmazás és a backend közötti kommunikáció során. Ennek a felépítésnek köszönhetően a Rails alkalmazások egyszerűen összekapcsolhatók más rendszerekkel és frontend megoldásokkal [2].

Végül, a Rails beépített tesztelési eszközei lehetőséget adnak az automatizált tesztelésre, amely hozzájárul a szoftverek magasabb megbízhatóságához és a hibamentes működés biztosításához a fejlesztés minden szakaszában. Az ilyen automatizált tesztelési funkciók szintén csökkentik a karbantartási költségeket, hiszen a kódminőség magasabb szinten tartható, és gyorsan azonosíthatók a hibák [1, 2].

3.1.2. PostgreSQL

A *PostgreSQL* egy *nyílt forráskódú, objektum-relációs adatbázis-kezelő rendszer*, amely széles körben ismert teljesítményéről, rugalmasságáról, és különösen alkalmas komplex adatstruktúrák és nagy adattömegek kezelésére. Támogatja a fejlett adattípusokat, bonyolult lekérdezéseket, és párhuzamos műveleteket, amelyek különösen hasznosak a Bus4U-hoz hasonló, valós idejű adatokat feldolgozó rendszerekben [3, 4].

A PostgreSQL előnyei a Bus4U számára:

- **Teljesítmény optimalizálása és bonyolult lekérdezések kezelése:** A PostgreSQL kiválóan kezeli a párhuzamos lekérdezéseket és optimalizált módon képes kezelni nagy mennyiségű adatot és komplex lekérdezési struktúrákat. Ez biztosítja az alkalmazás gyors, megbízható működését, és lehetőséget nyújt az adatbázis hatékony méretezésére és finomhangolására az adatmennyiség növekedésével [4].
- **Skálázhatóság:** Nagyobb adatmennyiség vagy felhasználói terhelés esetén a PostgreSQL támogatja a függőleges és vízszintes skálázást. Ezen túlmenően replikációval biztosítható a magas rendelkezésre állás, amely kritikus szempont a nagy terhelés alatt működő rendszerekben. Így a PostgreSQL hosszú távú, fenntartható teljesítményt és magas rendelkezésre állást biztosít, amely elengedhetetlen a Bus4U megbízható működéséhez [3].

3.1.3. Azure

Az *Azure* egy olyan nagyvállalati szintű, biztonságos és skálázható felhőplatform, amely számos lehetőséget kínál modern alkalmazások fejlesztéséhez és működtetéséhez. Az Azure infrastruktúrája révén biztosítható a folyamatos és megbízható működés, míg a platform rugalmassága lehetővé teszi az erőforrások hatékony kezelését és a skálázhatóságot [5, 6]. Az Azure használatával elkerülhetőek a hagyományos szerver infrastruktúrák fizikai

korlátai, és olyan erőforrások válnak elérhetővé, amelyek az alkalmazás igényei szerint rugalmasan méretezhetők.

Az Azure előnyei a Bus4U számára:

- **Skálázhatóság és rugalmasság:** Az Azure automatikusan alkalmazkodik a változó terheléshez, amely lehetővé teszi, hogy a rendszer gördülékenyen kezelje a megnövekedett felhasználói forgalmat. Ez az adaptív megközelítés egy modern, folyamatosan növekedő alkalmazás számára elengedhetetlen, különösen tömegközlekedési rendszerek esetében, ahol a felhasználói igények időben változhatnak [5].
- **Biztonság és adatvédelem:** Az Azure erőteljes adatvédelmi szabályokat és fejlett biztonsági protokollokat alkalmaz. Ezek közé tartozik a többfaktoros hitelesítés, a hálózati tűzfalak, valamint az adattitkosítás, amelyek biztosítják, hogy a Bus4U felhasználóinak adatai biztonságban legyenek. A platform megfelel a szigorú adatvédelmi szabványoknak, ami kulcsfontosságú egy olyan alkalmazás számára, amely szenzitív felhasználói adatokat kezel [6].

3.2. A közlekedési rendszerek és a jövőbeli trendek

A közlekedési rendszerek jövőbeli fejlődése alapvetően meghatározza az olyan alkalmazások, mint a Bus4U sikerét. Levinson és Krizek [7] szerint a közlekedés jövője radikális változások elé néz, amelyek magukban foglalják a forgalom csökkenését és az új mobilitási megoldások elterjedését. A közlekedés jövőjére vonatkozó előrejelzések hangsúlyozzák az automatizálást, a megosztott közlekedési módok és az okos városok integrálását, amelyek mind hozzájárulnak a fenntarthatóbb és hatékonyabb közlekedési hálózatok kialakításához. A Bus4U alkalmazás azzal, hogy valós idejű információkat biztosít a közlekedési rendszerről, közvetlenül hozzájárulhat ezen trendek előmozdításához.

Ceder [8] a közlekedési rendszerek tervezésében és üzemeltetésében alkalmazott eljárásokat és módszereket vizsgálja, különös figyelmet fordítva a tömegközlekedési rendszerek hatékonyságának és minőségének javítására. Az alkalmazás szempontjából kulcsfontosságú lehet, hogy a különböző típusú közlekedési eszközök integrálása és azok közötti interakciók optimalizálása révén a rendszer a lehető legjobb utazási élményt biztosítsa. Az automatizált jegyvásárlás és az utazási időszámítás optimalizálása mind hozzájárulhat a felhasználók elégedettségéhez.

3.3. Hasonló alkalmazások

A közlekedési alkalmazások, mint például a Budapest Go, jelentős hatással vannak a modern városi közlekedésre. A Budapest Go a magyarországi főváros tömegközlekedési rendszerének digitális megoldása, amely lehetővé teszi a felhasználók számára, hogy

egyszerűen és gyorsan intézzék utazásaik szervezését. Az alkalmazás számos funkcióval rendelkezik, amelyek javítják a felhasználói élményt.

A Budapest Go célja, hogy a város tömegközlekedését még hozzáférhetőbbé és felhasználóbarátabbá tegye. Az alkalmazás főbb jellemzői közé tartozik:

- Valós idejű menetrendek: Az alkalmazás folyamatosan frissíti a buszok, villamosok és metrók érkezési idejét, lehetővé téve a felhasználók számára, hogy pontosan tudják, mikor indul a következő járat.
- Online jegyvásárlás: A felhasználók az alkalmazáson belül jegyeket vásárolhatnak, ami gyorsítja a folyamatot és csökkenti a sorban állás szükségességét.
- Útvonaltervezés: Az alkalmazás lehetővé teszi a felhasználók számára, hogy megtervezzék utazásaikat, figyelembe véve a különböző közlekedési eszközöket és a lehető leghatékonyabb útvonalakat.
- Használati statisztikák: A Budapest Go elemzi a felhasználók utazási szokásait, lehetővé téve a szolgáltatók számára, hogy jobban megértsék a közlekedési igényeket és fejleszthessék szolgáltatásaikat.

3.3.1. Felhasználói élmény és innovációk

A Budapest Go nemcsak a közlekedési információk egyszerűsítésére szolgál, hanem innovatív megoldásokat is kínál a felhasználók számára. Az alkalmazás intuitív felhasználói felülete lehetővé teszi a könnyű navigációt, így még a technológiához kevésbé értő felhasználók is könnyen el tudnak igazodni.

3.3.2. Hatások a közlekedési rendszerre

A Budapest Go alkalmazás bevezetése jelentős hatással volt a főváros közlekedési rendszerére. A digitális jegyvásárlás elterjedése csökkentette a készpénzes tranzakciók számát, így gyorsabbá téve a közlekedési folyamatokat. Ezenkívül a valós idejű információk elérhetősége segít a felhasználóknak a jobb döntéshozatalban, ami hozzájárul a város közlekedési hatékonyságának növeléséhez.

A Budapest Go egy példa arra, hogyan lehet a digitális technológiát alkalmazni a városi közlekedés fejlesztésére. Az alkalmazás által kínált funkciók és a felhasználói élmény folyamatos javítása hozzájárul a közlekedési rendszerek modernizálásához és a fenntartható közlekedés népszerűsítéséhez.

4. fejezet

Rendszer specifikációi

4.1. Felhasználói követelmények

4.1.1. Felhasználói weboldal

A felhasználói weboldal bejelentkezés nélkül használható. A főoldalon lehetőség nyílik járatokra keresni dátum, idő és cél alapján. A menetrendeknél megtekinthetők a buszok és megállók órarendjei. A megállók oldalán az összes megálló térképen jelenik meg, és szűrést is lehet alkalmazni. Az élőben frissülő térképen a buszok pozícióját percről percre lehet követni. A jegyek oldalán listázva vannak az eddig megvásárolt jegyek QR-kódjai.

Fiókot létrehozni kizárólag a jegyvásárláshoz szükséges. A főoldalon, ha a felhasználó megtalálta a megfelelő járatot, ráléphet a vásárlás gombra. A fiók regisztrációjához meg kell adni:

- teljes név
- email cím
- új jelszó
- ellenőrző kód (amit a rendszer elküld a megadott email címre)

4.1.2. Admin felület

Az admin felület kizárólag bejelentkezéssel elérhető. Új fiókokat csak az adott cég vezetője tud létrehozni az "Alkalmazottak" oldalon. Az adminoknak lehetőségük van:

- a megállók listájának szerkesztésére
- az útvonalak összes tulajdonságának módosítására (meglátogatott megállók, indulási időpontok, jegyárak)

- a főoldalon elérhető statisztikák megtekintésére (eladott jegyek száma, megtett kilométer, bevétel)
- a buszok adatainak kezelésére (útadó, biztosítás, műszaki vizsga, buszok kivonása forgalomból, új buszok hozzáadása)

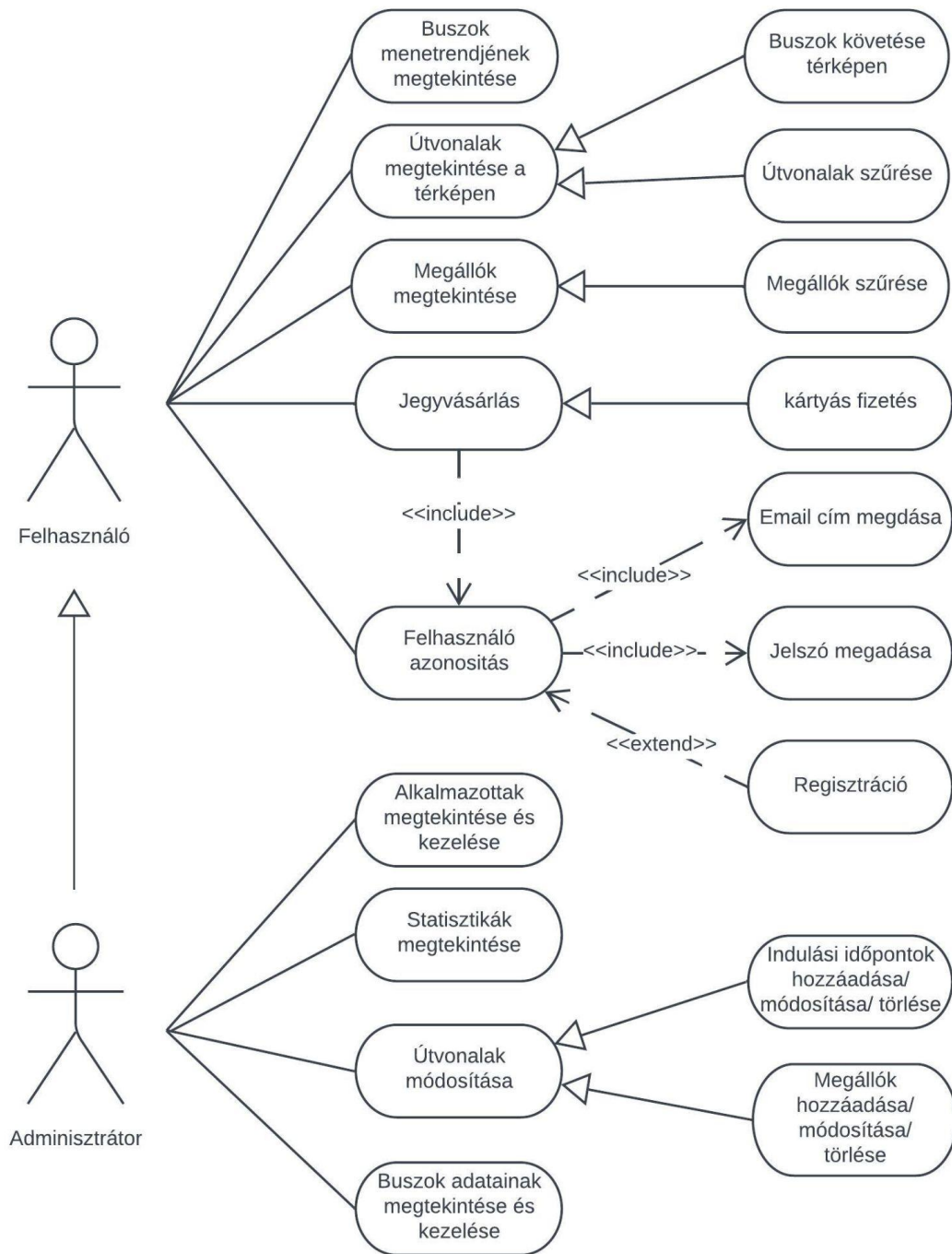
4.1.3. Mobil applikáció

A Flutter alapú mobil applikáció ugyanúgy működik, mint a felhasználóknak szánt weboldal, így ugyanazok a követelmények vonatkoznak erre is.

4.1.4. POS terminál alkalmazás

A POS terminál alkalmazás használatához szükség van egy sofőr szerepkörű admin fiókra. Bejelentkezés után lehetőség van egy adott busz követésének elindítására, valamint a jegyszkennelés működtetésére.

A rendszer használati eset diagramja az 4.1-es ábrán látható.



4.1. ábra. Use-case diagram a felhasználók és az adminisztrátorok lehetőségeiről

4.2. Rendszerkövetelmények

4.2.1. Funkcionális követelmények

A felhasználói felület minden oldalán van egy kis form, amelyet kitöltve le lehet kérni az adatbázisból az adott oldalon megjelenítendő adatokat. Ilyenre példa van a főoldal-

lon is, ahol meg kell adni az indulás helyét, a cél helyét (települést) és opcionálisan az ezekhez tartozó megállót, egy-egy lenyíló listából kiválasztva az adatokat. A települések betöltődnek az oldallal együtt, a megállók listája viszont csak a város kiválasztása után, mindig csak az adott településen találhatóak jelennek meg. Ezen kívül a főoldalon van egy dátumválasztó és egy időválasztó is, ezek alapértelmezetten az aktuális dátummal és idővel vannak kitöltve. Ha a form ki van töltve, a keresés gomb lenyomására az oldal elküldi a bevitt adatokat a backend szerverre, amely válaszként elküld egy listát a megfelelő utazási lehetőségekkel. Ezek az adatok JSON formátumban, HTTP protokollon keresztül közlekednek a frontend és backend között. A többi oldal is hasonlóan működik, először az adatokat kell megadni, aztán a gomb lenyomására pillanatok alatt megjelenik a képernyőn a szerver válasza a bevitt adatokra.

A különböző formokon kívül, a weboldalon található térképek is API hívásokkal működnek. A rendszer először betölti az alap térképet a MapBox API segítségével, aztán amint a felhasználó kiválasztotta a megtekinteni kívánt útvonalat a Menetrend vagy az Élő térkép lapon, a weboldal a saját adatbázisunkból kéri le a hozzá tartozó megállók koordinátáit, ezeket megjeleníti a térképen, végül ismét a MapBox API-t használva útvonaltervet generáltat. Ez úgy működik, hogy a rendszer elküldi a megállók listáját és visszakap egy sok pontból álló új listát a MapBox-tól, ezeket összekötve sok kis vonallal, kirajzolódik a részletes útvonal a térképen.

4.2.2. Nem-funkcionális követelmények

Szervezéssel kapcsolatos követelmények ♦

- **Programozási nyelvek:** Ruby, Dart, HTML, CSS, JavaScript
- **Keretrendszerek:** Ruby on Rails, Flutter, Vue.js
- **IDE:** Visual Studio Code
- **Verziókövetés:** Git és GitHub
- **Adatbázis:** PostgreSQL
- **Cloud szolgáltatás:** Azure
- **Webhoszt:** Netlify
- **Csoportos kommunikáció:** Slack

Termékkel kapcsolatos követelmények ♦ A felhasználói weboldal és az admin felület használatához modern, internetezésre alkalmas eszközre van szükség, legalább 2018-as vagy frissebb böngészőkkel:

- **Chrome:** 87 vagy újabb
- **Firefox:** 78 vagy újabb
- **Safari:** 14 vagy újabb
- **Edge:** 88 vagy újabb

Az applikáció használatához internettel és GPS antennával rendelkező okostelefon vagy tablet szükséges, a következő minimális követelményekkel:

- **Operációs rendszer:** Android 5.0 vagy iOS 8.0 és újabb
- **Szabad tárhely:** minimum 120 MB
- **Memória:** 516–1024 MB

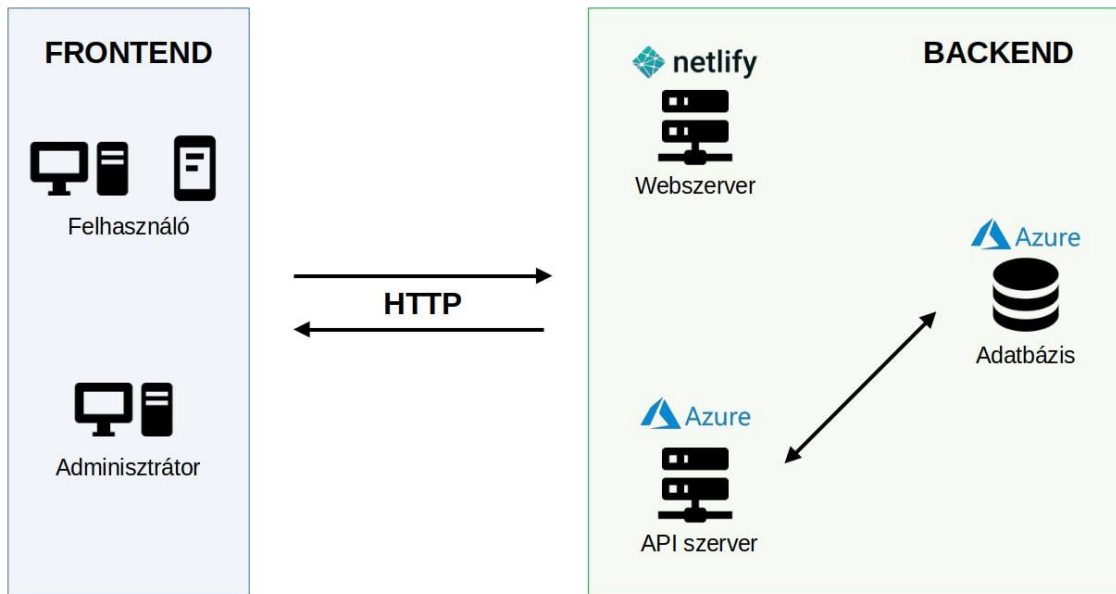
A POS terminál alkalmazás futtatásához egy Sunmi V2 PRO POS terminál ajánlott. Alternatívaként, ha nem áll rendelkezésre, bármely kamerával és GPS antennával ellátott, minimum Android 5.0 vagy iOS 8-at futtató eszköz is használható:

- **Szabad tárhely:** legalább 100 MB
- **Memória:** minimum 510 MB

5. fejezet

A rendszer részletes leírása

Rendszerünk egy Ruby on Rails keretrendszerben írt API-ból, egy Vue.js alapú webes felhasználói- és admin felületből, egy Flutter alkalmazásból a felhasználók számára, valamint egy Flutter keretrendszerben készült POS terminál alkalmazásból. Az API a REST architektúrát követi, lehetővé téve a különböző erőforrások, például buszok, járatok és jegyek kezelését. A felhasználói és admin weboldal Vue-ban készült, a Vuetify elemek felhasználásával, ami lehetővé tette az egyszerű és hatékony felhasználói felületek gyors fejlesztését. A Vue.js keretrendszer előnye, hogy komponensekből áll, így átlátható és jól rendszerezett webalkalmazás felépítést tesz lehetővé. Ezen kívül a Vuetify komponenskészlete megkönnyítette az UI elemek létrehozását, gyorsítva ezzel a fejlesztési folyamatot. Az alkalmazás Flutterben készült, Dart programozási nyelvet használva, amely lehetővé teszi, hogy ugyanazt a kódbázist használjuk iOS és Android platformokra is, így jelentősen csökkentve a fejlesztési időt és költségeket. A POS terminál alkalmazása is Flutter keretrendszerben készült, biztosítva a hatékony és egységes megoldást a különböző platformok számára. A Vue.js alapú weboldalakat, illetve a felhasználóknak szánt Flutter alkalmazást Portik Szabolcs kollegám mutatja be részletesebben egy másik dolgozat keretein belül.



5.1. ábra. A rendszer architektúra diagramja

5.1. Az API felépítése

A Bus4U API a REST architektúrát követi, és számos végpontot biztosít a különböző erőforrások kezeléséhez, mint például:

- **Buszok kezelése:** buszok létrehozása, frissítése, törlése.
- **Járatok menedzselése:** járatok lekérése és ütemezése.
- **Jegyek és bérletek:** jegyek és bérletek vásárlása és kezelése.

5.2. Endpointok részletes leírása

5.2.1. GET /get_stations_by_city

Ez az endpoint lehetővé teszi a megállók listájának lekérését egy adott város alapján. A kérés a következő paramétert használja:

- **city_uid:** A város egyedi azonosítója.

Példa kérés:

GET `https://api.bus4u.online/v1/get_stations_by_city?city_uid=CTY_N11XH`

Válasz formátum:

```
{
  "stations": [
    {
      "station_uid": "STT_EGAVX",
      "name": "Sapientia",
      "address": "Str. Calea Sighisoarei nr. 2",
      "longitude": "24.59883",
      "latitude": "46.52339"
    },
    {
      "station_uid": "STT_6C03Q",
      "name": "Dedeman",
      "address": "Bul. 1 Decembrie nr. 189",
      "longitude": "24.59941",
      "latitude": "46.52678"
    }
  ]
}
```

5.3. Adatkezelés és adatbázis kapcsolat

Az API végpontjai a **PostgreSQL** adatbázissal kommunikálnak. Az adatbázis tranzakciókat az **ActiveRecord** ORM segítségével kezeljük, amely lehetővé teszi az adatok egyszerű és hatékony lekérését, beszúrását és frissítését.

5.3.1. Adatbázis táblák

A PostgreSQL adatbázisunk a következő táblákból áll:

- **Companies**: Azon cégek adatai, akikkel szerződést kötött a Bus4U.
- **Admins**: Különböző cégekhez tartozó adminisztrátorok adatai. Szerepük lehet “boss” vagy “driver”.
- **Buses**: A cégekhez tartozó autóbuszok adatai.
- **Routes**: A cégek saját útvonalai.
- **Stations**: A rendszerben található buszmegállók összessége.
- **Route_stations**: Egy cég adott útvonalán keresztülhaladó megállók.

- **Timetables:** Egy útvonal megállójának indulási időpontjai és ára nap szerinti felosztásban.
- **Cities:** Az összes város, amelyen áthaladnak útvonalak.
- **Users:** A Bus4U felhasználói a fontosabb adataikkal.
- **Email_verifications:** Az adott email címekhez tartozó regisztrációkor felhasználó kódok.
- **Tickets:** A felhasználókhoz tartozó jegyek.
- **Payments:** Stripe fizetések eltárolása.

5.4.1. Jira

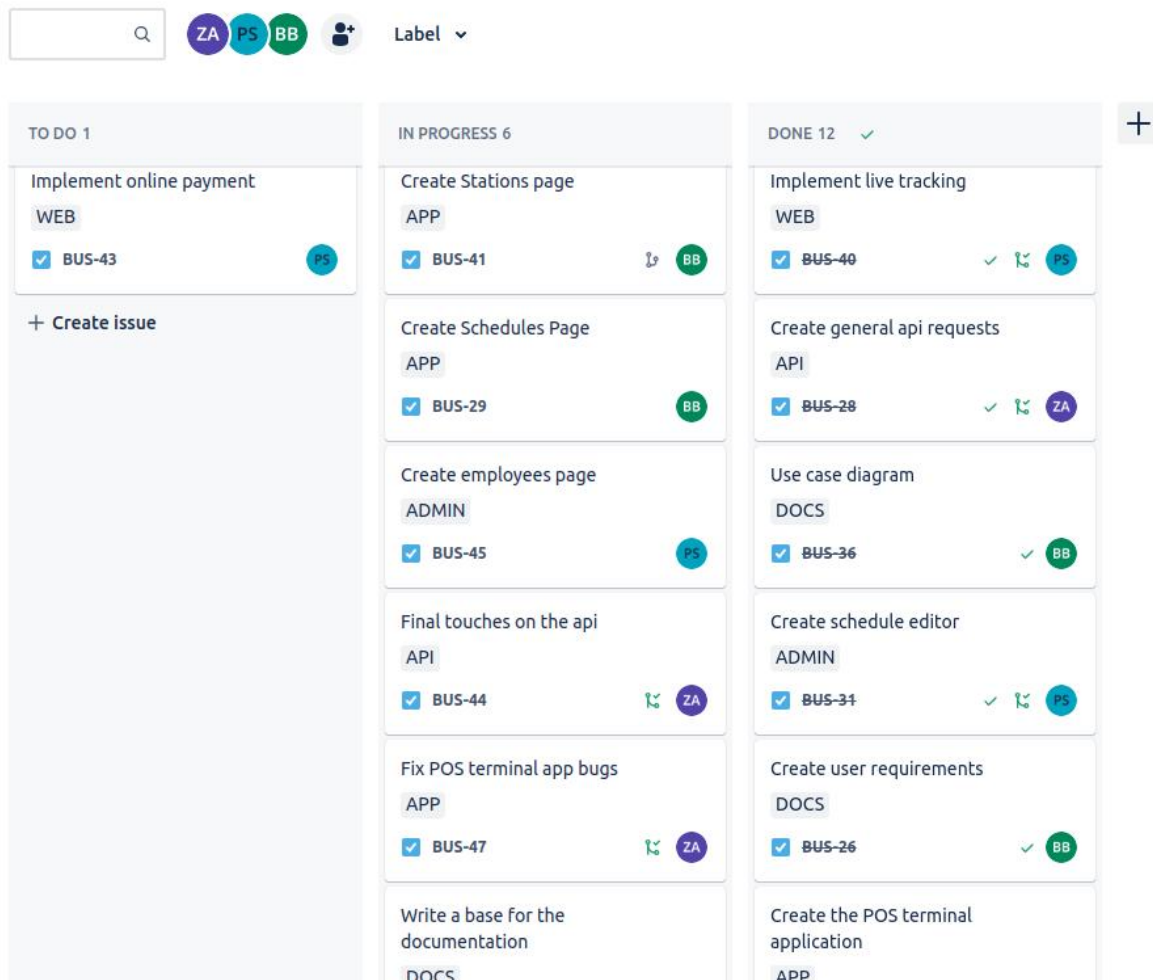
A projektmenedzsment során a **Jira** kanban boardját használtuk, amely az Atlassian által fejlesztett projekt-követő platform. A Jirát választottuk, mert egy olyan hatékony eszköz, amely egyszerre letisztult, ám funkciókban komplex, így könnyedén igazítható a projekt igényeihez.

A Kanban tábla három fő oszlopból állt:

- **To do**: Ide kerültek azok a feladatok, amelyeket még nem kezdtünk el.
- **In progress**: Ebben az oszlopban azok a feladatok szerepeltek, amelyek éppen folyamatban voltak.
- **Done**: Az elvégzett feladatok ebbe az oszlopba kerültek, jelezve azok befejezését.

Minden feladatot hozzárendeltünk egy konkrét tulajdonoshoz, valamint címkékkel láttunk el, amelyek megjelölték, hogy a munka épp a rendszer melyik komponensére vonatkozik: **API**, **felhasználói felület**, **admin felület**, vagy **mobil alkalmazás**. A feladatok ilyen címkézése segített abban, hogy pontosan tudjuk, ki mivel foglalkozik, illetve milyen állapotban van a fejlesztés különböző területein végzett munka.

BUS board



5.3. ábra. A Jira kanban board áttekintése

5.4.2. Git és GitHub

A projekt forráskódjának verziókezelését a **Git** verziókezelő rendszer segítségével oldottuk meg, míg a kód tárolására a **GitHub** platformot használtuk. A GitHub lehetőséget biztosított a pull request-ek hatékony kezelésére, amelyekkel minden változtatást egyszerűen tudtunk jóváhagyni. Ez a megközelítés a kód visszakövethetőségét és dokumentálását is jelentősen megkönnyítette, hozzájárulva a kód minőségének javításához és a fejlesztési folyamat átláthatóságához.

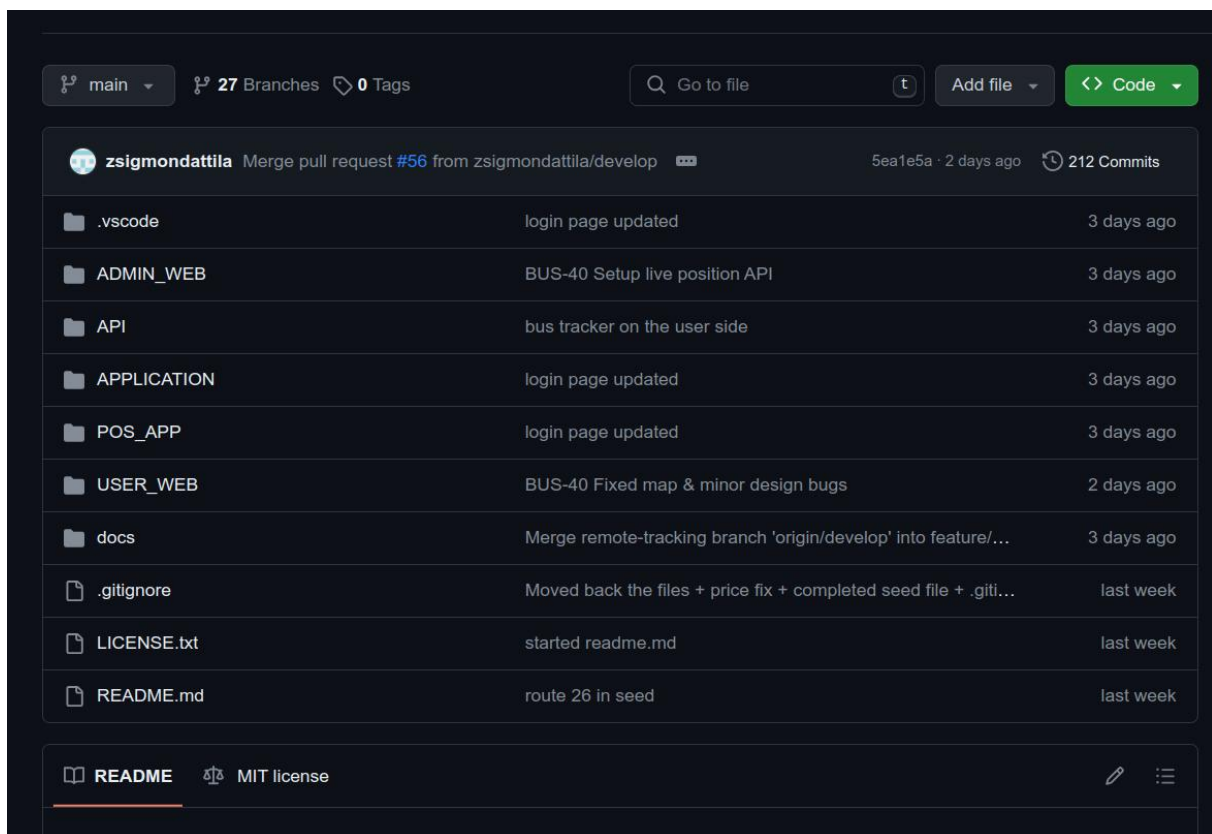
A projekt struktúráját három fő **branch** köré szerveztük:

- **main branch**: Ebbe az ágba kizárólag a már teljesen tesztelt és stabil kódrészletek kerültek be, amelyeket időszakosan publikáltunk. Ez az ág szolgált a hivatalos, működőképes verzió tárolására.
- **develop branch**: A develop branch-et a fejlesztés és tesztelés helyszínéként hasz-

náltak. Az ide feltöltött kódok még tesztelés alatt álltak, de a fejlesztési idő alatt mindig a develop verzióját futtattuk az Azure környezetben, ezzel biztosítva, hogy az aktuális fejlesztések folyamatosan elérhetők és tesztelhetők legyenek.

- **feature branchek**: Az új funkciók fejlesztését különálló, ún. feature branchekben végeztük, amelyek a konkrét feladatokra vagy kódrészletekre koncentráltak. Például kisebb munkaigényű kódok, mint az általános API lekérések fejlesztése külön branchekben történt, míg az összetettebb funkciókat külön feature branch-ekben dolgoztuk ki.

A branch-ek ilyen felosztása lehetővé tette a párhuzamos fejlesztést, és biztosította, hogy a különböző fejlesztési szakaszok kódjai elkülönítve és rendezetten legyenek tárolva. A GitHub pull request funkciójának köszönhetően minden változtatást felül tudtunk vizsgálni, így minden módosítás előtt biztosak lehettünk annak minőségében és kompatibilitásában.



5.4. ábra. A GitHub repository áttekintése

Ezek az eszközök hozzájárultak a projekt sikeres és hatékony fejlesztéséhez, elősegítve a feladatok egyértelműsítését, priorizálását és a csapat tagjai közötti együttműködést.

6. fejezet

Infrastruktúra

6.1. Azure infrastruktúra áttekintése

A Bus4U rendszer a Microsoft Azure felhőalapú platformján került telepítésre. Az Azure szolgáltatások skálázhatóságot és megbízhatóságot biztosítanak a rendszer számára. A telepítéshez az alábbi főbb szolgáltatásokat használtam:

- **Azure Virtual Machines (VM):** A szerver hosztolása.
- **Azure Database for PostgreSQL:** Az adatbázis-szolgáltatás biztosítása.

6.2. Domain konfiguráció

A rendszer az **api.bus4u.online** domainen érhető el, amelyet a Cloudflare platformján konfiguráltam. A domain hozzárendelése során a virtuális gép (VM) IP címét használtam. A következő beállításokat végeztem el:

- **DNS beállítások:** A Cloudflare-en elvégeztem a domain névhez tartozó A és CNAME rekordok konfigurálását.
- **SSL tanúsítványok:** A HTTPS alapú kapcsolat biztonságának érdekében a Cloudflare szolgáltatásait alkalmazzuk, amelyek védelmet nyújtanak a felhasználók számára.

6.3. Adatbázis telepítése és konfigurációja

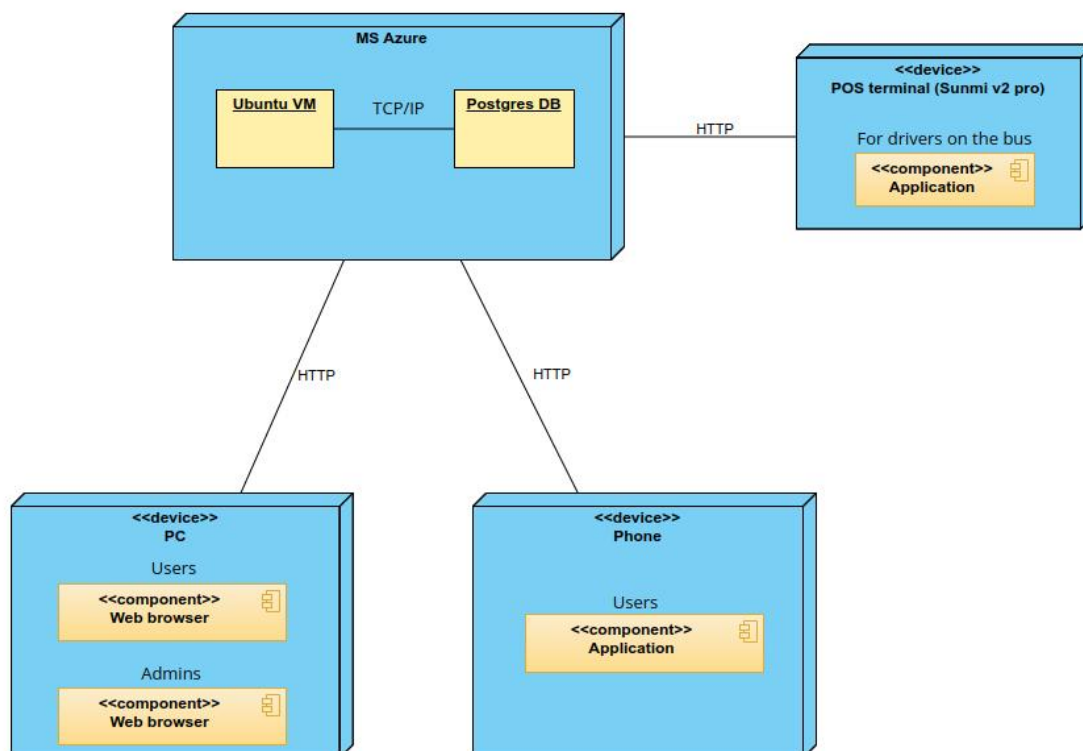
A rendszer adatbázisa egy **Azure Database for PostgreSQL** szolgáltatáson keresztül került telepítésre, amely lehetővé teszi az adatbázis automatikus méretezését és a folyamatos biztonsági mentések készítését. A következő beállításokat alkalmazzuk:

- **Titkosított kapcsolatok:** A PostgreSQL adatbázis SSL tanúsítványokat használ a biztonságos kommunikációhoz.
- **Backup és recovery:** Automatikus biztonsági mentések kerülnek tárolásra az adatvesztés elkerülése érdekében.

6.4. Szerver telepítése

A szerver az **Azure Virtual Machines** szolgáltatásban került telepítésre, ahol a Ruby on Rails alapú API-t futtatjuk. A telepítés során a következő lépéseket hajtottuk végre:

- **Környezeti változók beállítása:** A szükséges konfigurációs változók (pl. adatbázis kapcsolatok) a Ruby on Rails beépített credentials funkciójában lettek biztonságosan tárolva és kezelve.
- **Verziókezelés:** Az **asdf** verziókezelő használatával a Ruby és egyéb nyelvek verzióit könnyen kezelhetjük, ami megkönnyíti a fejlesztési környezet fenntartását.
- **CI/CD pipeline:** Az alkalmazás automatikus telepítése és frissítése a GitHub Actions integrációjával valósult meg.



6.1. ábra. Telepítési diagram

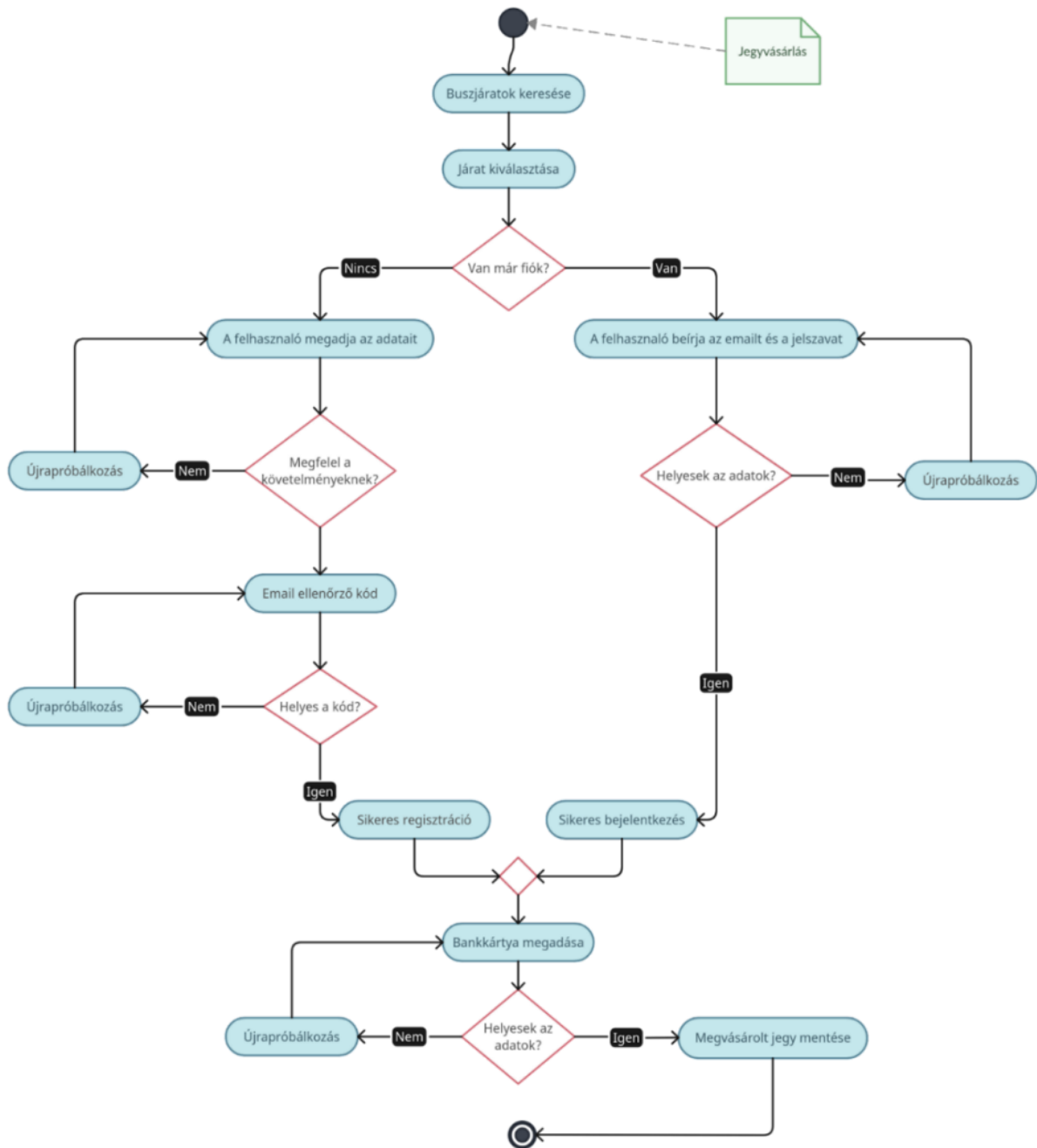
7. fejezet

Az alkalmazás működése

7.1. Felhasználói felület

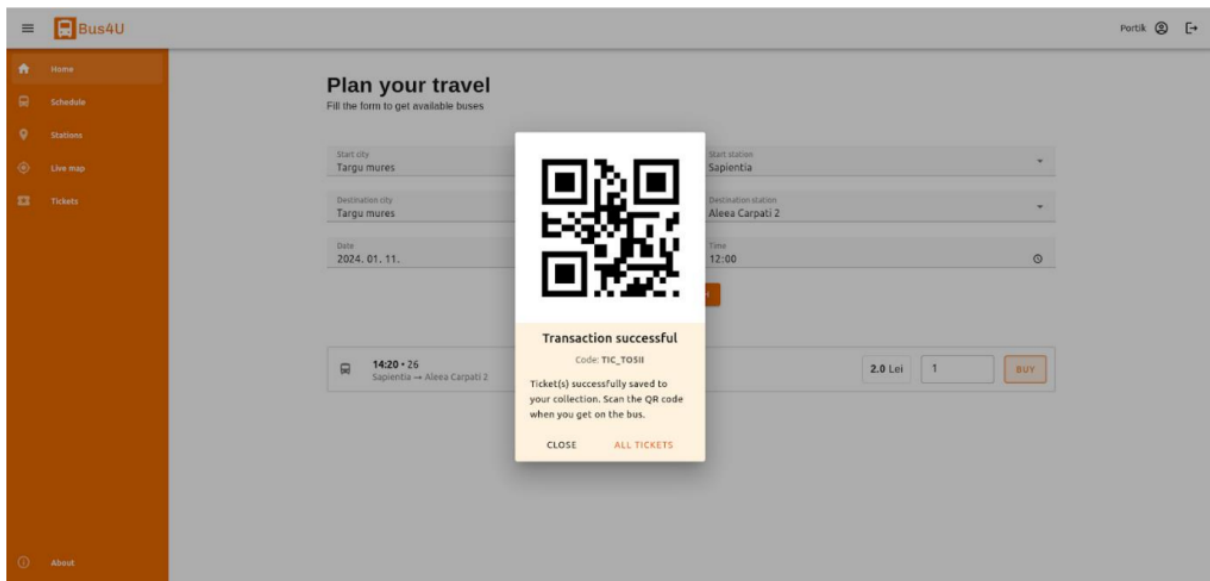
7.1.1. Főoldal

A weboldalra rákeresve a felhasználót a kezdőoldal fogadja, ahol lehetőség van egy utazás megtervezésére. Ki kell választani a kiindulási helyet és megállót, majd a célt, az utazás dátumát és időpontját. Ekkor a rendszer lekéri a szerverről a megadott megállók közötti, az adott napon, a megadott időpont után közlekedő járatokat és azok jegyárát. Minden egyes járatnál van egy vásárlás gomb, amellyel a felhasználó megszerezheti a jegyet vagy jegyeit az adott buszjáratra, amennyiben be van jelentkezve, ellenkező esetben a rendszer átirányítja a bejelentkezéshez.



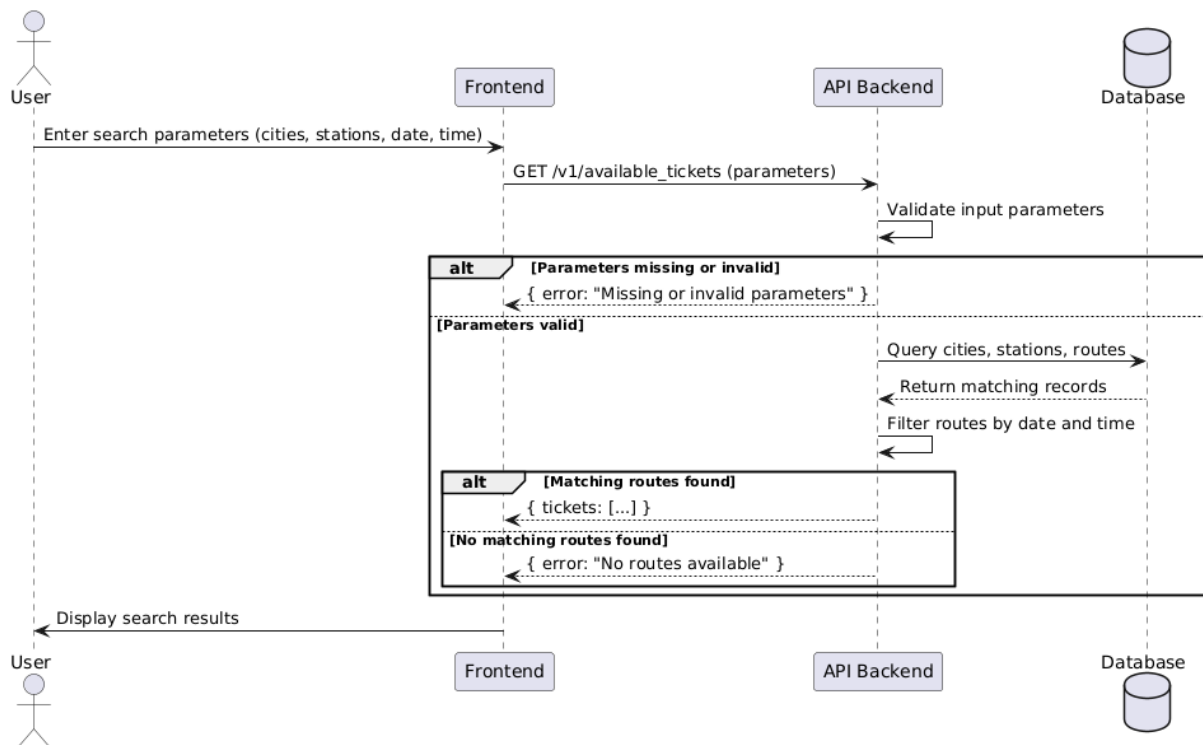
7.1. ábra. Aktivitás diagram az utazás tervezésének folyamatáról

Ahogy az ábrán is látható, miután a felhasználó megtalálta a megfelelő buszjáratot és rálépett a jegy(ek) megvásárlására, a rendszer átirányítja őt a bejelentkezéshez, itt ha van már fiókja, bejelentkezhet a meglévő adataival, amennyiben nincs, úgy átléphet a regisztrációs oldalra, ahol meg kell adnia a nevét, email címét, és egy új jelszót. A rendszer minden adatot ellenőriz és ha valamelyik hibás (pl. érvénytelen email cím vagy túl rövid jelszó) akkor egy hibaüzenetet ír ki. Ez addig ismétlődik, amíg minden adat megfelelő lesz, aztán automatikusan küldünk egy ellenőrző kódot a megadott e-mailre, amelyet a felhasználó be kell írjon a webes felületen megjelenő szövegmezőbe.



7.2. ábra. Megvásárolt jegy

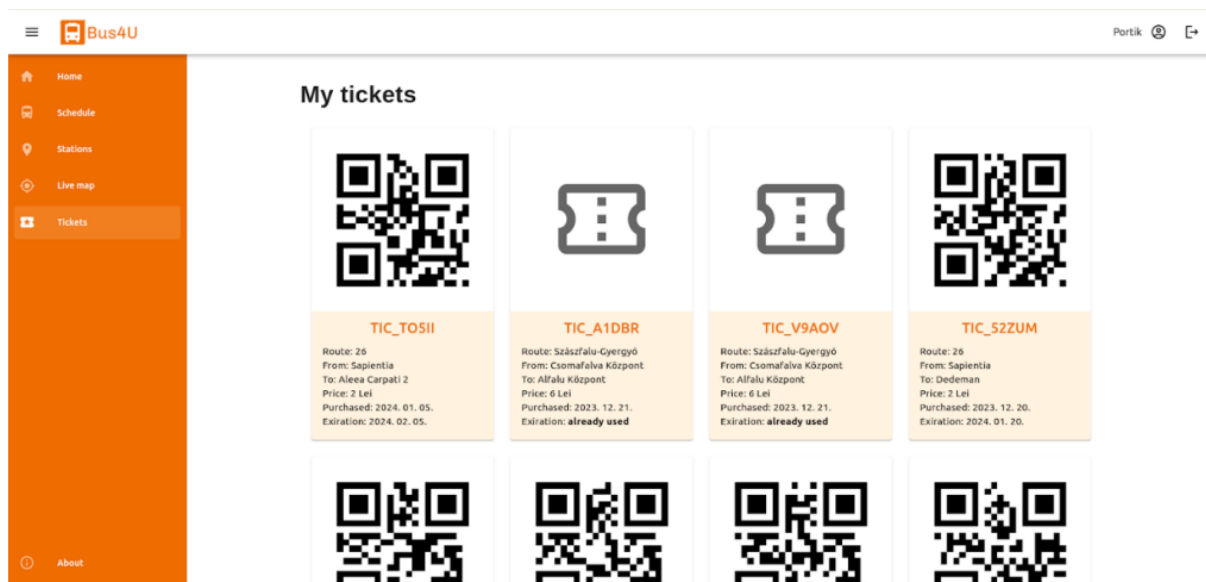
Ha ez helytelen, ez is addig ismétlődik, amíg minden rendben lesz, majd a rendszer át-irányítja a felhasználót a bankkártyás fizetési oldalra, ami jelenleg nincs implementálva, ez a jövőbeli tervek között szerepel. Ezután már csak az következik, hogy a rendszer el-menti a megvásárolt jegy kódját, amelyet a felhasználó a Jegyek oldalon talál meg szöveges és QR-kódos formátumban.



7.3. ábra. Szekvencia diagram az útvonaltervezés folyamatáról

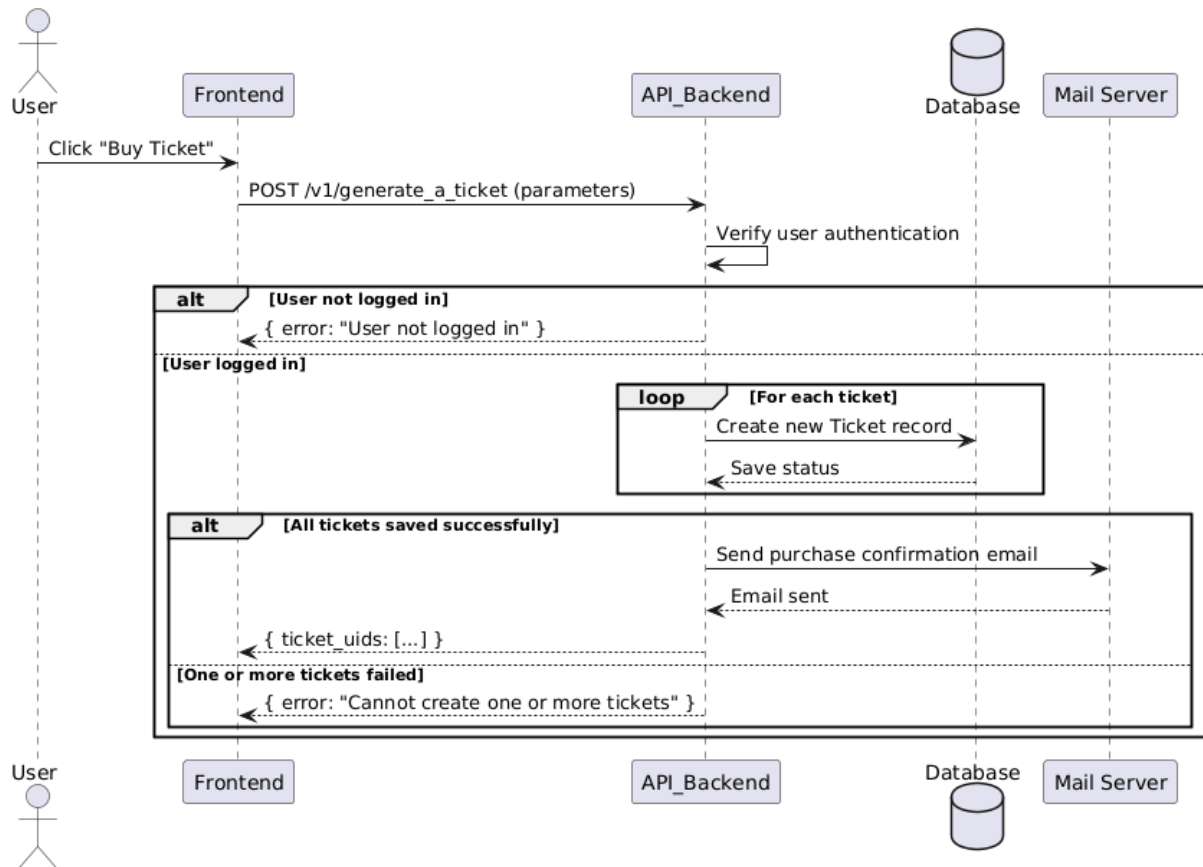
7.1.2. Jegyek

A megvásárolt jegyeket a rendszer elmenti a gyűjteménybe, ami a Jegyek oldalon található, így a jegyek bármikor visszakereshetők és felhasználhatóak lesznek, kivéve ha lejár az egy hónapos érvényesség.



7.4. ábra. Felhasználó megvásárolt jegyei

A jegyen megjelenített adatok között van a járat neve, a kiinduló- és célállomás, az ár, valamint a vásárlás és az érvényesség dátumai. A már elhasznált jegyek megvannak jelölve így azokat nem lehet többször felhasználni.



7.5. ábra. Szekvencia diagram a jegyvásárlás folyamatáról

7.1.3. Menetrend

A következő oldal a menetrend, amely hasonlóan a buszmegállóknak kihelyezett táblázatokhoz, megjeleníti a kiválasztott buszjárat indulási időpontjait, az adott megállóból. A mi megoldásunk viszont tartalmaz emellett egy térképet is, ahol meg lehet nézni a járat útvonalát, az érintett megállókat és az útvonalon közlekedő buszok élő pozícióját is.

Schedule

Select a station and a bus to view the departure timetable and the route marked on the map

City
Marosvásárhely

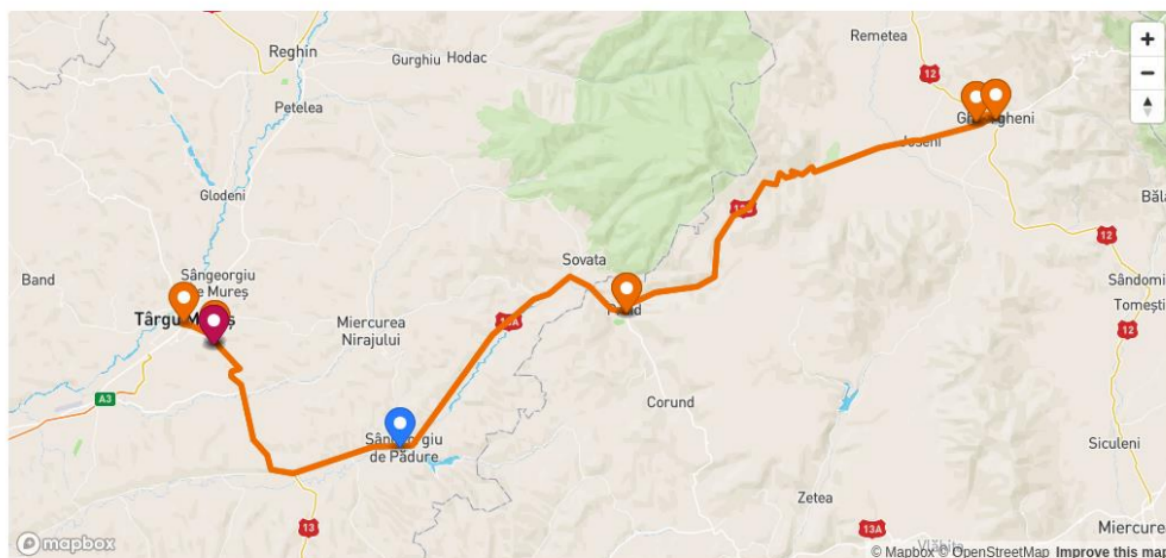
Station
Dedeman

Bus
Gyergyó-Marosvásárhely

SEARCH

Departure times for station: Dedeman and bus: Gyergyó-Marosvásárhely

Monday	07:45	19:45
Tuesday	07:45	19:45
Wednesday	07:45	19:45
Thursday	07:45	19:45
Friday	07:45	19:45



☒ Current location

☒ Live bus location

☒ Stations

7.6. ábra. Listázott menetrend és az útvonal térképe

7.1.4. Megállók és térkép

Ez a két oldal a fent említett térképes funkciókat mutatják meg külön-külön, egyszerűbben. Egyik megjeleníti a térképen az összes megállót, és ezek között szűrést is biztosít város vagy buszjárat szerint. A másik oldalon pedig egy-egy útvonalon közlekedő buszok élő pozícióját lehet követni, miután kiválasztottuk a várost és az útvonalat. Ez a funkció abban segít, hogy ne maradjunk le, ha egy busz siet, illetve ne várakozzunk feleslegesen a

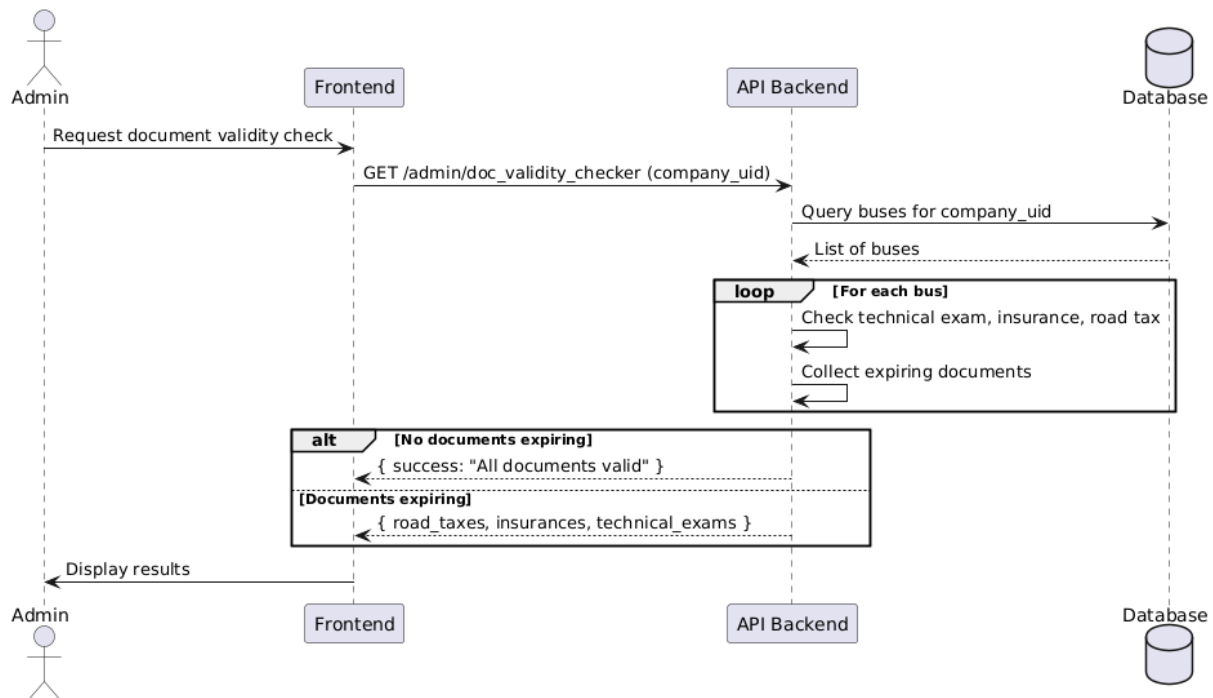
megállóban, ha egy busz késik, így időt lehet vele spórolni.

7.2. Adminisztrációs felület

Az admin felület csak bejelentkezéssel elérhető, ehhez egy admin rangú felhasználói fiók szükséges, amely egy adott céghez van hozzárendelve. A cégek adatai el vannak különítve, így minden admin csak a saját cégéhez tartozó adatokat láthatja és módosíthatja.

7.2.1. Kezdőlap

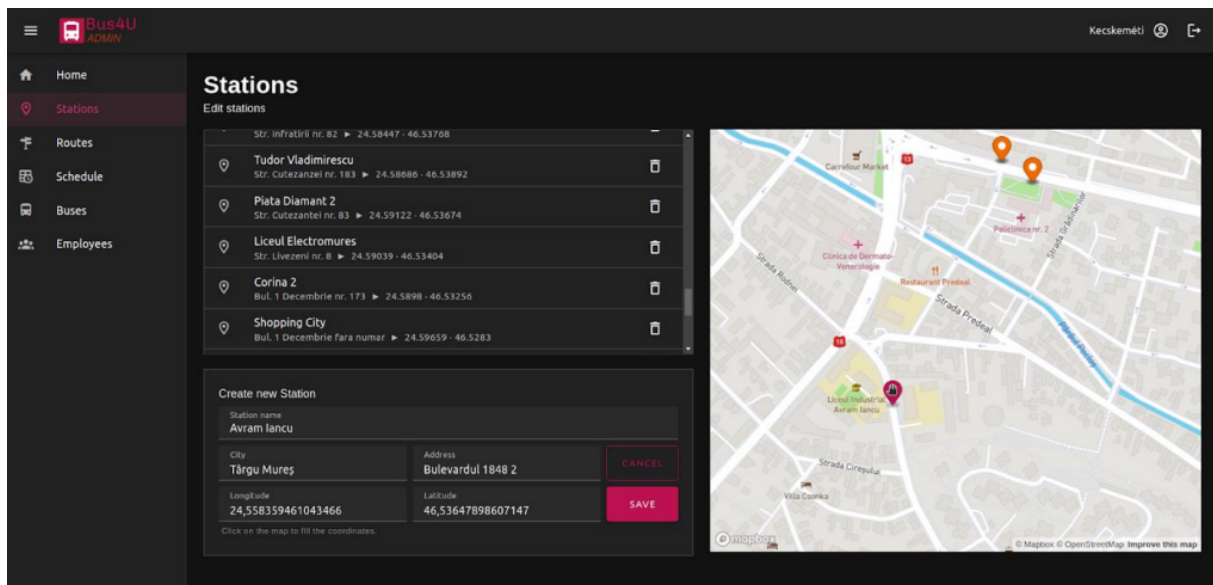
Belépés után egy összegző oldal jelenik meg amelyen látható egy kis statisztika a regisztrált felhasználók számával, a megvásárolt jegyek számával és a bevétellel, mindez havi, évi és mindenkor bontásban. Itt jelennek meg az esetleges figyelmeztetések és értesítések is, például a buszok lejárt útdíja vagy a közelőg műszaki vizsgálat.



7.7. ábra. Szekvencia diagram a dokumentumok ellenőrzéséről

7.2.2. Megállók

A Megállók oldalon létre lehet hozni új megállókat, megtekinteni meglévőket a térképen vagy törölni ezeket. Új megálló hozzáadásához meg kell adni a megálló nevét, a címet és a koordinátákat. A néven kívül, a többi adatot automatikusan kitölti a rendszer, ha a térképen a megálló helyére kattint a felhasználó, de manuálisan is meg lehet adni.



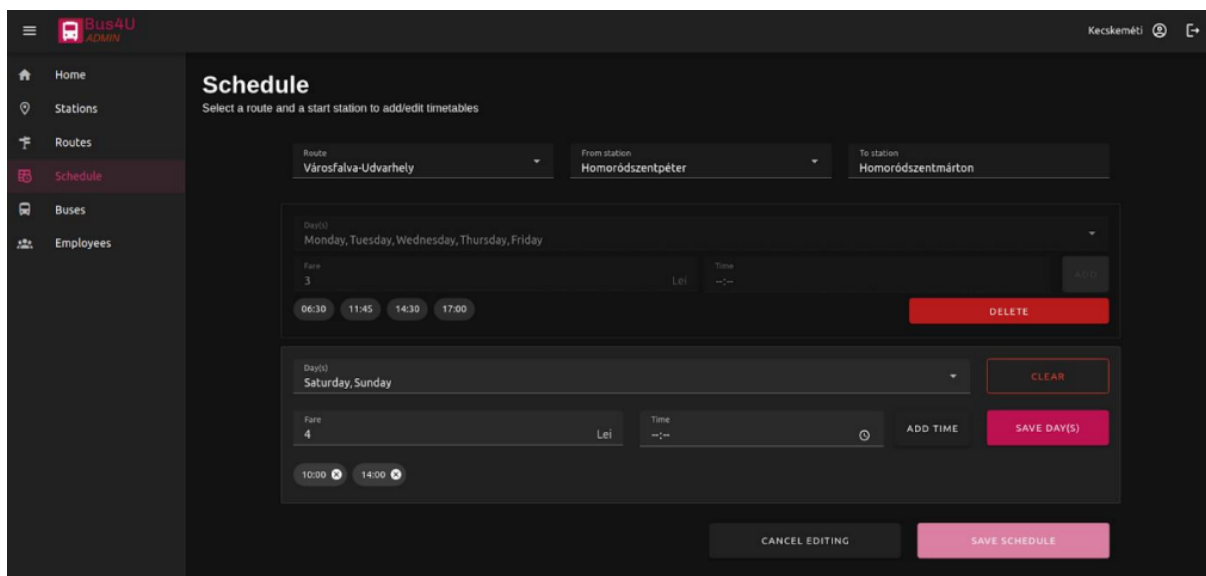
7.8. ábra. Megállók létrehozása/törlése

7.2.3. Útvonalak

Ezen az oldalon lehet felépíteni útvonalakat a létrehozott megállókból és itt lehet megadni az alap jegyárat is, abban az esetben, ha a jegyek ára nem függ a megtett távolságtól és időtől, hanem mindig és mindenhol ugyanannyi, ez esetben a fizetés az útvonalhoz lesz hozzárendelve, nem a különböző megállókhoz.

7.2.4. Menetrend

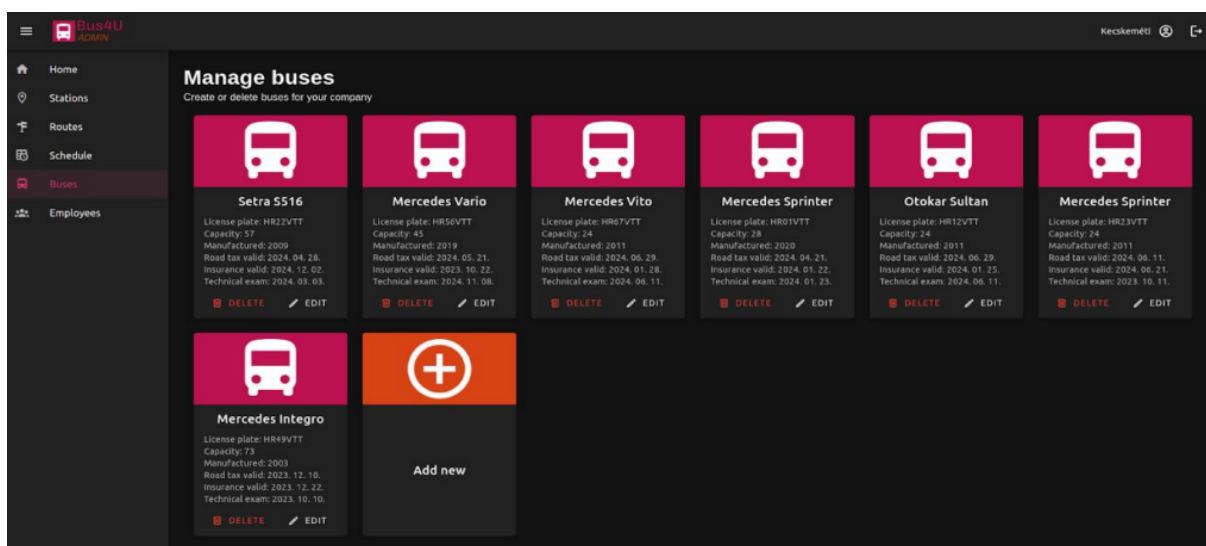
A következő oldal a Menetrend, ahol a létrehozott útvonalak megállóihoz lehet órarendeket hozzárendelni. Itt meg kell adni az órarendre érvényes napokat, a következő megállóig szükséges jegyárat és az indulási időpontokat. Egy megállóhoz hozzá lehet adni több ilyen órarendet is, ha különböző napokra különböző órarend érvényes. Így el lehet választani a hétvégét a hétköznapoktól, ha mások az indulási időpontok, vagy ha például hétvégén drágább a jegy.



7.9. ábra. Menetrend létrehozása/törlése

7.2.5. Buszok és alkalmazottak

Az utolsó két oldalon a buszokat és az alkalmazottakat lehet kezelni. Hasonlóan működnek, mindkettőn található egy lista a meglévő elemekkel, ahol bármelyiket lehet módosítani vagy törölni, illetve újat is lehet létrehozni. A buszoknál meg kell adni a márkát, a rendszámot, a férőhelyek számát, a gyártási évet, valamint az útdíj, a biztosítás és a műszaki engedély lejáratát dátumát. Új alkalmazott hozzáadásánál meg kell adni a nevet, e-mail címet, beosztást, telefonszámot, lakcímet (opcionális), és egy új jelszavát.

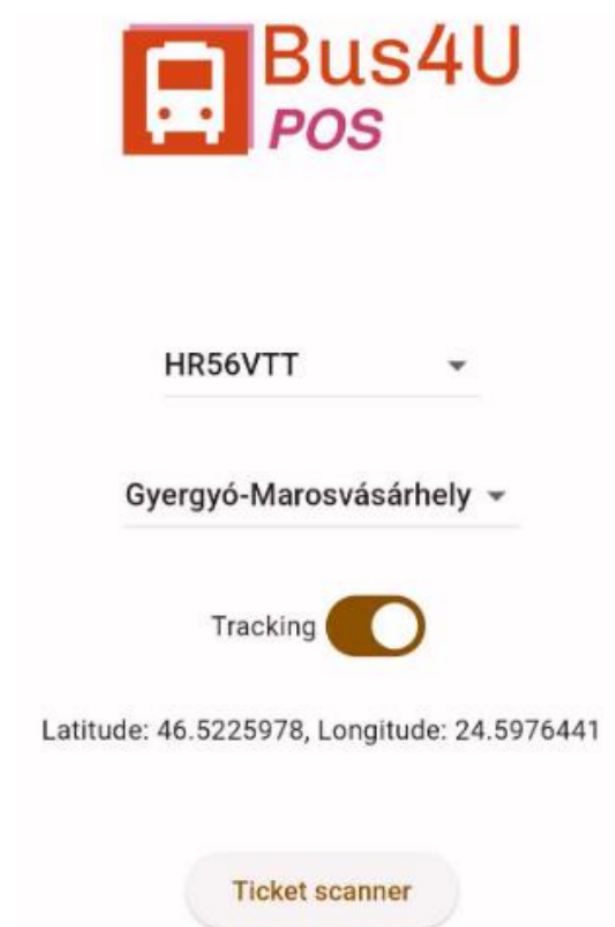


7.10. ábra. Buszok és alkalmazottak kezelése

7.3. POS terminál alkalmazása

7.3.1. GPS oldal

Miután a sofőr bejelentkezett, ezen az oldalon lehetőség nyílik kiválasztani az aktuális buszt, amelyiket jelenleg a sofőr vezet, illetve azt az útvonalat, amelyet éppenséggel végrehajt. Található egy kapcsoló, amellyel el lehet indítani az autóbusz lokációjának követését. Ez félpercenként küldi fel az API-ra a busz lokációját, ezáltal lehet a weboldalon és az alkalmazásban élőben követni a busz helyzetét. Legalul egy Ticket scanner gomb fedezhető fel, amely átirányít a jegyszkennelő oldalra



7.11. ábra. Busz kiválasztása és GPS követés

7.3.2. Jegyszkennelő oldal

Ez az oldal a kamerát jeleníti meg. Itt kell beszkennelni a jegyet. Szkennelés után egy felugró ablak jelenik meg 5 másodpercig, három üzenetet hordozhat: Ha sikeresen érvényesítve van a jegy, akkor zöld háttérrel kiírja hogy sikeres érvényesítés és jó utazást kíván az alkalmazás. Ha a jegy már érvényesítve volt, vagy érvénytelen a QR kód, ezt egy piros hibaüzenettel jelzi. Illetőleg ha az API valamilyen oknál fogva leáll, akkor ennek a

hibájáról is üzenetet kapunk.



7.12. ábra. Jegy szkennelése

8. fejezet

A rendszer tesztelése

8.1. Tesztelési környezet

A tesztelést egy helyi fejlesztési környezetben végeztem. A szerver a következő konfigurációval működött:

- **Operációs rendszer:** Ubuntu 22.04 LTS
- **Szerver:** Ruby on Rails 7.0.4
- **Adatbázis:** PostgreSQL 14
- **Tesztelő eszköz:** Postman v10.0

8.2. Funkcionális tesztelés

A rendszer funkcionális teszteléséhez a Postman eszközt használtam. A Postman lehetőséget nyújt HTTP-kérések küldésére, valamint a különböző API végpontok működésének, válaszainak és adatstruktúráinak ellenőrzésére. Az alábbiakban bemutatom a tesztelés folyamatát és az eredményeket.

A tesztelést egy helyi fejlesztési környezetben végeztem, ahol a szerver a következő címen volt elérhető:

`http://localhost:3000/api/v1/`

A Postman segítségével különböző HTTP metódusokat (GET, POST, PUT, DELETE) használtam a végpontok tesztelésére. Az egyes végpontokat a rendszer megfelelő válaszai alapján ellenőriztem.

8.2.1. Végpontok tesztelése

Az alábbi végpontok tesztelését szeretném kiemelni:

- **GET** `api_url/v1/get_stations_by_city` – Lekéri az állomásokat a megadott város alapján.
- **GET** `/v1/get_departure_times_for_station_in_route` – Indulási időpontok lekérése egy adott járatra és állomásra.
- **GET** `/v1/get_available_tickets` – Elérhető jegyek lekérdezése az adott kiindulási és célállomásra.
- **GET** `/v1/get_bus_locations_by_route` – A buszok aktuális helyzetének lekérése egy adott járatra.
- **GET** `/v1/tickets_of_user` – Az aktuális felhasználó jegyeinek lekérése.
- **GET** `/v1/admin/document_validity_checker` – Dokumentum érvényességének ellenőrzése adminisztrátorok számára.
- **GET** `/v1/admin/statistics` – Statisztikai adatok lekérdezése egy adott cég számára.

8.2.2. Tesztelési példa

Például a GET metódus használatával kértem le az állomásokat egy megadott város alapján:

```
GET api.bus4u.online/v1/get_stations_by_city?city_uid=CTY_N11XH
```

A szerver válasza a következő JSON formátumban érkezett:

```
{
  "stations": [
    {
      "station_uid": "STT_EGAVX",
      "name": "Sapientia",
      "address": "Str. Calea Sighisoarei nr. 2",
      "longitude": "24.59883",
      "latitude": "46.52339"
    },
    {
      "station_uid": "STT_6C03Q",
```

```

        "name": "Dedeman",
        "address": "Bul. 1 Decembrie nr. 189",
        "longitude": "24.59941",
        "latitude": "46.52678"
    }
]
}

```

Ezzel a lekérdezéssel sikeresen megkaptam az adatokat, amelyeket a rendszer helyesen dolgozott fel.

8.2.3. Tesztelési eredmények

A tesztelés során az összes végpontot felseztem Postman-re, és mindegyik megfelelően működött. Az egyes végpontok válaszai megfeleltek az elvárt adatstruktúrának, és a rendszer helyesen kezelte a kéréseket.

8.3. Teljesítménytesztelés

A teljesítménytesztelés célja a rendszer stabilitásának és skálázhatóságának értékelése volt nagy terhelés alatt. A vizsgálat során a JMeter eszközt használtam, amely képes különböző mennyiségű egyidejű felhasználói kérések szimulálására.

A JMeter mintavételi eredményei alapján az alábbi teljesítménymetrikák és azok jelentőségeit vizsgáljuk az API teljesítményével kapcsolatban:

8.3.1. Teljesítmény-metrikák

KIEGÉSZÍTENI AZ ÚJRATESZTELT ADATOKKAL

8.4. Biztonsági tesztelés

A biztonsági tesztelés során különös figyelmet fordítottam a következő sebezhetőségek elkerülésére:

- **SQL injection** – Input validálás és előkészített SQL lekérdezések használata.
- **Cross-site scripting (XSS)** – HTML entitások kódolása a felhasználói bemeneleknél.
- **Cross-site request forgery (CSRF)** – CSRF tokenek használata minden kényes műveletnél.

8.5. Összegzés

A tesztelési eredmények alapján a rendszer minden funkciója megfelelően működik, a teljesítménye kielégítő, és a biztonsági mechanizmusok jól védik az API-t a lehetséges támadások ellen. A további tesztelések célja az API skálázhatóságának javítása, valamint az integrációs lehetőségek vizsgálata más külső rendszerekkel.

9. fejezet

Összefoglalás

Ez a fejezet összefoglalja a Bus4U rendszer API-jának tervezését, megvalósítását és tesztelését, valamint bemutatja a legfontosabb megállapításokat és javaslatokat a jövőbeli fejlesztésekhez.

A dolgozat célja egy hatékony busz menedzselő rendszer API létrehozása volt, amely lehetővé teszi a felhasználók számára a buszjáratok, jegyek és bérletek kezelését, valamint a rendszeres frissítéseket és karbantartásokat. A projekt során a következő lépéseket tettük:

1. Követelmények és tervezés: A rendszer funkcionális és nem funkcionális követelményeinek részletes elemzése és a legfontosabb funkciók prioritásának meghatározása. A követelmények alapján a rendszer architektúráját és az API felépítését terveztük meg.

2. API megvalósítása: Az API endpointok megvalósítása során külön figyelmet fordítottunk a RESTful architektúra elveinek betartására, biztosítva ezzel a könnyű használhatóságot és a skálázhatóságot. A végpontok különböző erőforrásokat (buszok, járatok, jegyek, bérletek) kezelnek, lehetővé téve a felhasználók számára, hogy egyszerűen hozzáférjenek az információkhoz és végrehajtsanak műveleteket.

3. Tesztelés: A rendszer tesztelését Postman segítségével végeztük, ahol különböző kéréseket küldtünk az API végpontokhoz, hogy ellenőrizzük azok működését, a válaszok helyességét és a teljesítményt. A tesztelési folyamat során sikerült azonosítani és javítani a hibákat, így a rendszer stabil és megbízható lett.

4. Deployment: A rendszer telepítése az Azure felhőszolgáltatásra történt, ahol az API az `api.bus4u.online` domainen érhető el. A PostgreSQL adatbázis és a virtuális gép (VM) külön szolgáltatásként működik, biztosítva a skálázhatóságot és a megbízhatóságot.

5. Jövőbeli fejlesztések: A jövőbeli fejlesztések között szerepel a rendszer funkcionálisának bővítése, például új szolgáltatások bevezetése, a felhasználói élmény javítása érdekében. További cél a rendszer biztonságának növelése, például a hitelesítési és jogosultsági mechanizmusok fejlesztésével.

Összességében a Bus4U API megvalósítása sikeres volt, és a dolgozat során végrehajtott lépések hozzájárultak a hatékony és megbízható busz menedzselő rendszer kialakí-

tásához. A projekt során szerzett tapasztalatok és tanulságok értékes alapot jelentenek a jövőbeli fejlesztésekhez és a hasonló rendszerek tervezéséhez.

Irodalomjegyzék

- [1] M. Hartl, *Ruby on Rails Tutorial: Learn Web Development with Rails*. Addison-Wesley Professional, 2016.
- [2] O. Fernandez, *The Rails Way*. Addison-Wesley Professional, 2017.
- [3] K. Douglas and S. Douglas, *PostgreSQL: Up and Running*. O'Reilly Media, 2015.
- [4] J. Murrell, *PostgreSQL 12 High Availability Cookbook*. Packt Publishing, 2019.
- [5] B. Wilder, *Cloud architecture patterns: using Microsoft Azure*. O'Reilly Media, Inc., 2012.
- [6] M. Collier and R. Shahan, *Microsoft Azure Essentials-Fundamentals of Azure*. Microsoft Press, 2015.
- [7] D. M. Levinson and K. J. Krizek, *The End of Traffic and the Future of Transport*. Network Design Lab, 2016.
- [8] A. Ceder, *Public Transit Planning and Operation*. Elsevier, 2016.

A függelék

Függelék

függelék